

Forward without **forward**

Contents

- [Motivation](#)
- [Proposal](#)
- [Revision History](#)

Motivation

Consider the following example:

```
template <class X, class Y>
decltype(auto) foo(X&& x, Y&& y) {
    return std::forward<X>(x)(std::forward<Y>(y));
}
```

It's a simple function, but it takes some time to figure out what's actually going on. It's even more awkward when we consider the same function in lambda form (although this awkwardness has been partially alleviated with the adoption of [P0428](#)):

```
// C++17
auto bar = [](auto&& x, auto&& y) {
    return std::forward<decltype(x)>(x)(std::forward<decltype(y)>(y));
};

// C++2a
auto bar = []<class X, class Y>(X&& x, Y&& y) {
    return std::forward<X>(x)(std::forward<Y>(y));
};
```

This verbosity and lack of clarity lead many people to actually use a *macro* to make forwarding shorter:

```
#define FWD(x) std::forward<decltype(x)>(x)

template <class X, class Y>
decltype(auto) foo(X&& x, Y&& y) {
    return FWD(x)(FWD(y));
}

auto bar = [](auto&& x, auto&& y) {
    return FWD(x)(FWD(y));
};
```

A search on GitHub shows over 100 such definitions. That is substantially clearer, but are we really going to suggest that people use a macro?! Let's not. Forwarding references even got their name from the fact that they are intended to be `std::forward()`-ed, so it's awkward when the expected and typical usage is just so very verbose. The body of the lambda with `std::forward` is 73 characters, the body with `FWD` is 23. It is the contention of this proposal that those extra characters harm readability and don't help anybody.

Code that forwards is just so much more verbose than code that doesn't. But forwarding in itself doesn't justify the verbosity or resulting complexity. Unlike `std::move` and `std::ref` which are used to do non-typical things and deserve to be visible markers for code readability and understanding, `std::forward` is much more typical in these contexts and does not need as much of a sign post. We can do better.

Proposal

This paper would like to see a shorter way to forward arguments and proposes non-overloadable unary `operator>>`, where `>>expr` is defined as `static_cast<decltype(expr)&&>(expr)`. This addition would make it easier to write and, more importantly, read code that uses forwarding.

Note that `>>x`, where `x` is not an lvalue reference, is also equivalent to `std::move(x)`.

Examples demonstrating the improvement:

C++17	This proposal
<pre>template <class X, class Y></pre>	<pre>template <class X, class Y></pre>

<pre>decltype(auto) foo(X&& x, Y&& y) { return std::forward<X>(x)(std::forward<Y>(y)); } auto bar = [](auto&& x, auto&& y) { return std::forward<decltype(x)>(x)(std::forward<decltype(y)>(y)); };</pre>	<pre>decltype(auto) foo(X&& x, Y&& y) { return (>>x)(>>y); } auto bar = [](auto&& x, auto&& y) { return (>>x)(>>y); };</pre>
---	---

Implementing `std::apply` (code from [cppreference](#))

<pre>namespace detail { template <class F, class Tuple, std::size_t... I> constexpr decltype(auto) apply_impl(F&& f, Tuple&& t, std::index_sequence<I...>) { return std::invoke(std::forward<F>(f), std::get<I>(std::forward<Tuple>(t))...); } } // namespace detail template <class F, class Tuple> constexpr decltype(auto) apply(F&& f, Tuple&& t) { return detail::apply_impl(std::forward<F>(f), std::forward<Tuple>(t), std::make_index_sequence< std::tuple_size_v<std::decay_t<Tuple>>>{}); }</pre>	<pre>namespace detail { template <class F, class Tuple, std::size_t... I> constexpr decltype(auto) apply_impl(F&& f, Tuple&& t, std::index_sequence<I...>) { return std::invoke(>>f, std::get<I>(>>t)...); } } // namespace detail template <class F, class Tuple> constexpr decltype(auto) apply(F&& f, Tuple&& t) { return detail::apply_impl(>>f, >>t, std::make_index_sequence< std::tuple_size_v<std::decay_t<Tuple>>>{}); }</pre>
---	---

Delaying invocation (assuming pack expansion in init-capture - P0780)

<pre>template <class F, class... Args> auto delay_invoke(F&& f, Args&&... args) { return [f=std::forward<F>(f), args=std::forward<Args>(args)...]() -> decltype(auto) { return std::invoke(f, args...); }; }</pre>	<pre>template <class F, class... Args> auto delay_invoke(F&& f, Args&&... args) { return [f=>>f, args=>>args...]() -> decltype(auto) { return std::invoke(f, args...); }; }</pre>
---	--

Overload set as function object (assuming abbreviated lambda - P0573)

<pre>[] (auto&&... args) => f(std::forward<decltype(args)>(args)...) [&](args...) => obj.bar(std::forward<decltype(args)>(args)...) </pre>	<pre>[] (auto&&... args) => f(>>args...) [&](args...) => obj.bar(>>args...) </pre>
--	--

This proposal does not suggest deprecating `std::forward()`. There is nothing actually wrong with using it, and moreover there is another functionality of this function template that is quite important: forwarding expressions to a *different* type. This isn't common, but is more important to highlight as different from simple forwarding. This extension allows for making simple forwarding look simple, while making complex forwarding look complex.

Unary `>>` is ill-formed today so this is purely a language extension.

Impact on Compile Times

There are two ways that you can forward a variable: you can use `std::forward` or you can use `static_cast` directly (as this proposal's forwarding operator does):

```
template <class X, class Y>
decltype(auto) foo(X&& x, Y&& y) {
    return std::forward<X>(x)(std::forward<Y>(y)); // with std::forward
    return static_cast<X&&>(x)(static_cast<Y&&>(y)); // with static_cast, exactly equivalent
}
```

Louis Dionne [rewrote Boost.Hana](#) to use `static_cast` instead of `std::forward` after tests showed a 14% speed-up in compile times for template-heavy code. Granted, this is *very* specific use-case, but it does suggest a nice side benefit that a forwarding operator would provide.

Implementation Experience

An implementation of this proposal for gcc has been graciously provided by Bastien Penavayre on [github](#), along with a version of the [compiler explorer](#).

Revision History

Changes from r0: Dropped the proposal of using `>>` as a *capture-default*, in favor of simply proposing allowing pack expansion in *init-*

capture (see P0780). Added reference to P0428 for forwarding from generic lambdas, included compile time stats about `std::forward` vs `static_cast`, corrected some grammar and incorrect syntax in examples.